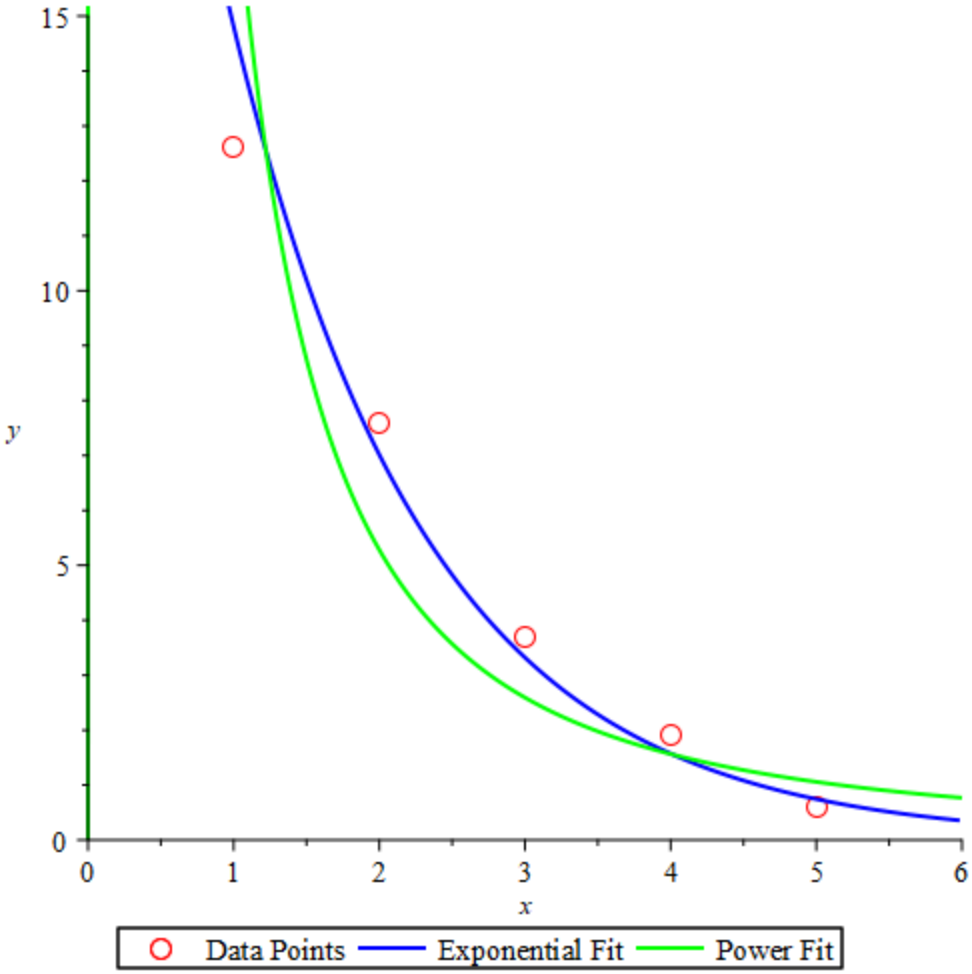
Hari Patel Extra Credit

1.

```
xdata := [1, 2, 3, 4, 5] :
ydata := [12.6, 7.6, 3.7, 1.9, 0.6] :
N := 5 :
# Transform to linear: ln(y) = ln(C) + A*x
Ylog := [seq(ln(ydata[i]), i = 1 ..N)] :
X := xdata :
# Solve for linear regression coefficients: ln(y) = A*x + ln(C)
SumX := add(X[i], i = 1 ..N) :
SumX2 := add(X[i]^2, i = 1 ..N) :
SumYlog := add(Ylog[i], i = 1 ..N) :
SumXY := add(X[i]*Ylog[i], i = 1 ..N) :
A_exp := (N*SumXY - SumX*SumYlog) / (N*SumX2 - SumX^2) :
lnC_exp := (SumYlog - A_exp*SumX) / N :
C_exp := exp(lnC_exp) :
# Transform to linear: ln(y) = ln(C) + A*ln(x)
Xlog := [seq(ln(xdata[i]), i = 1 ..N)] :
SumXlog := add(Xlog[i], i = 1 ..N) :
SumXlog2 := add(Xlog[i]^2, i = 1 ..N) :
SumXYlog := add(Xlog[i]*Ylog[i], i = 1 ..N) :
A_pow := (N*SumXYlog - SumXlog*SumYlog) / (N*SumXlog2 - SumXlog^2) :
lnC_pow := (SumYlog - A_pow*SumXlog) / N :
C_pow := exp(lnC_pow) :
with(plots) :
# Points
p_points := pointplot([seq([xdata[i], ydata[i]], i = 1 ..N)], symbol = circle, symbolsize = 15, color = red) :
# Exponential fit
p_exp := plot(C_exp*exp(A_exp*x), x = 0 ..6, y = 0 ..15, color = blue, thickness = 2) :
# Power fit
p_pow := plot(C_pow*x^A_pow, x = 0 ..6, y = 0 ..15, color = green, thickness = 2) :
# Combine plots
display(p_points, p_exp, p_pow, legend = ["Data Points", "Exponential Fit", "Power Fit"]);
```


15
10
5
0
y
0
1
2
3
4
5
6
x
Data Points
Exponential Fit
Power Fit

2.

```
f := x -> piecewise(x <= 0, x + 1, x > 0, x - 1):

# --- Continuous Fourier series coefficients ---
N := 4: # number of terms
a0 := evalf(int(f(x), x = -1..1)/2);
a := n -> evalf(int(f(x)*cos(n*Pi*x), x = -1..1));
b := n -> evalf(int(f(x)*sin(n*Pi*x), x = -1..1));
a_vals := [seq(a(n), n = 1..N)];
b_vals := [seq(b(n), n = 1..N)];
# --- Truncated Fourier series function ---
F_approx := x -> a0/2 + add(a(n)*cos(n*Pi*x) + b(n)*sin(n*Pi*x), n = 1..N);
# --- Plot continuous truncated Fourier series ---
plots:-display(
  plot(f(x), x = -3..3, color = blue, thickness = 2, legend = "f(x)"),
  plot(F_approx(x), x = -3..3, color = red, style = line, thickness = 2, legend = "Truncated Fourier series")
);
# --- Discrete Fourier series ---
h := 0.2:
xdata := [-1, -0.8, -0.6, -0.4, -0.2, 0, 0.2, 0.4, 0.6, 0.8]:
ydata := [seq(f(xdata[i]), i = 1..nops(xdata))]:
Npoints := nops(xdata);
a0_d := evalf((1/Npoints)*add(ydata[i], i = 1..Npoints));
an_d := n -> evalf((2/Npoints)*add(ydata[i]*cos(n*Pi*xdata[i]), i = 1..Npoints));
bn_d := n -> evalf((2/Npoints)*add(ydata[i]*sin(n*Pi*xdata[i]), i = 1..Npoints));
# Discrete series function
F_discrete := x -> a0_d/2 + add(an_d(n)*cos(n*Pi*x) + bn_d(n)*sin(n*Pi*x), n = 1..N);
# --- Plot discrete Fourier series ---
plots:-display(
  plot(f(x), x = -1..1, color = blue, thickness = 2, legend = "f(x)"),
  plot(F_discrete(x), x = -1..1, color = green, style = line, thickness = 2, legend = "Discrete Fourier series")
);
```

$$a0 := 0.$$

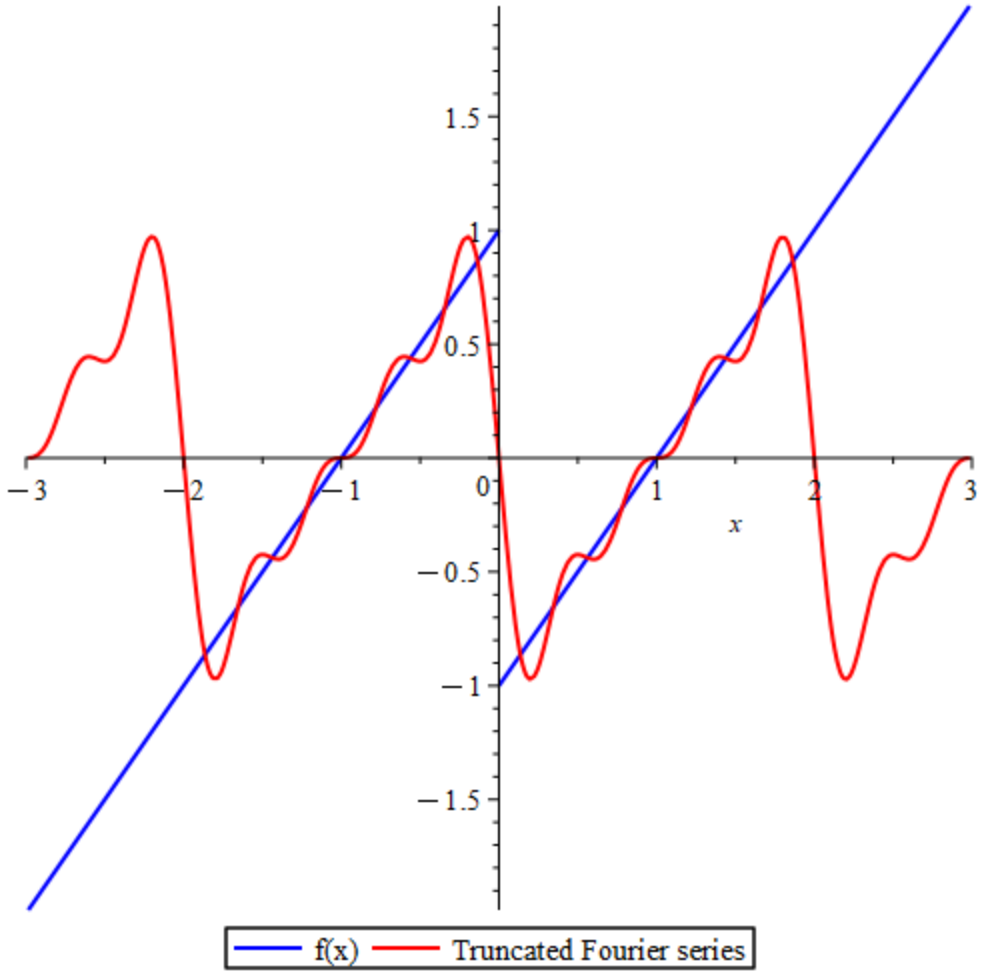
$$a := n \mapsto evalf\left(\int_{-1}^{1} f(x) \cdot \cos(n \cdot \pi \cdot x)\, dx\right)$$

$$b := n \mapsto evalf\left(\int_{-1}^{1} f(x) \cdot \sin(n \cdot \pi \cdot x)\, dx\right)$$

$$a\_vals := [0., 0., 0., 0.]$$

$$b\_vals := [-0.6366197722, -0.3183098861, -0.2122065907, -0.1591549430]$$

$$F\_approx := x \mapsto \frac{a0}{2} + add\left(a(n) \cdot \cos(\pi \cdot n \cdot x) + b(n) \cdot \sin(\pi \cdot n \cdot x), n = 1..N\right)$$



$$Npoints := 10$$

$$a0\_d := 0.1000000000$$

$$an\_d := n \mapsto evalf\left(\frac{2 \cdot add\left(ydata_i \cdot \cos\left(n \cdot \pi \cdot xdata_i\right), i = 1..Npoints\right)}{Npoints}\right)$$

$$bn\_d := n \mapsto evalf\left(\frac{2 \cdot add\left(ydata_i \cdot \sin\left(n \cdot \pi \cdot xdata_i\right), i = 1..Npoints\right)}{Npoints}\right)$$

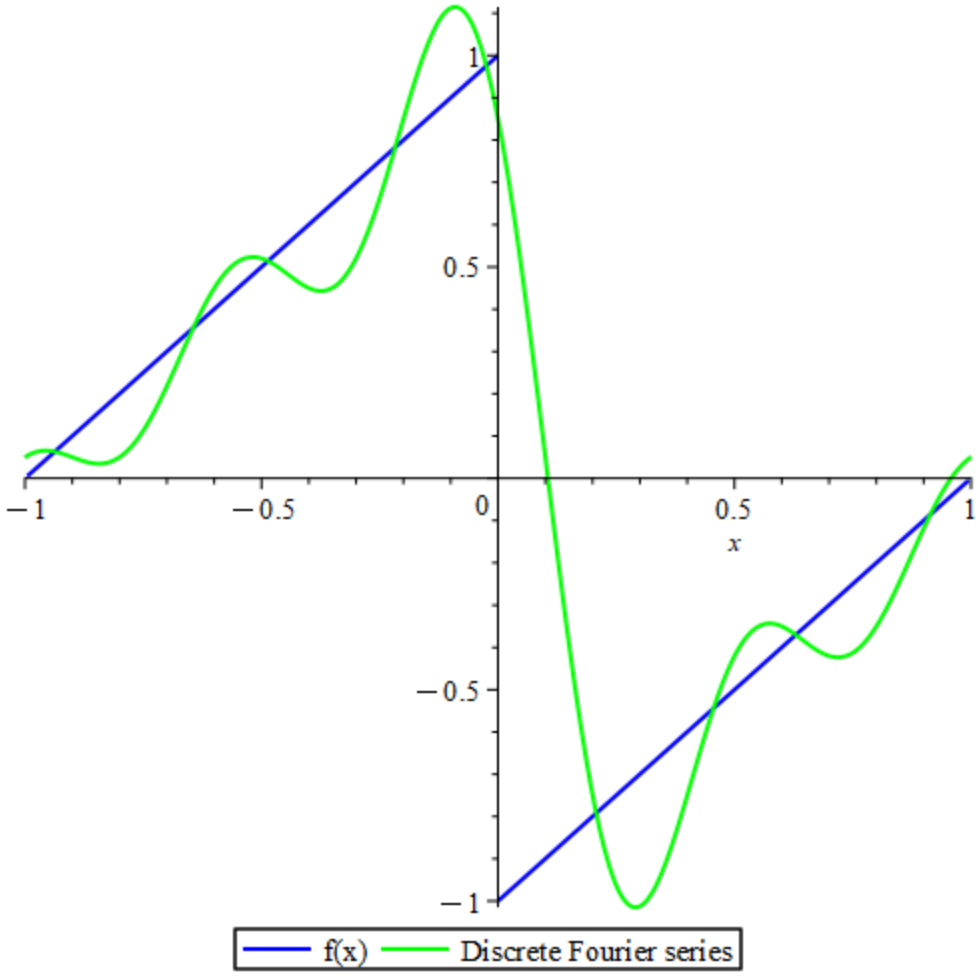
$$Npoints := 10$$

$$a0\_d := 0.1000000000$$

$$an\_d := n \mapsto evalf\left(\frac{2 \cdot add\left(ydata_i \cdot \cos\left(n \cdot \pi \cdot xdata_i \right), i = 1 \ldots Npoints \right)}{Npoints} \right)$$

$$bn\_d := n \mapsto evalf\left(\frac{2 \cdot add\left(ydata_i \cdot \sin\left(n \cdot \pi \cdot xdata_i \right), i = 1 \ldots Npoints \right)}{Npoints} \right)$$

$$F\_discrete := x \mapsto \frac{a0\_d}{2} + add\left(an\_d(n) \cdot \cos\left(\pi \cdot n \cdot x \right) + bn\_d(n) \cdot \sin\left(\pi \cdot n \cdot x \right), n = 1 \ldots N \right)$$



3

```
> # Define the functions g1 and g2
  g1 := (x1, x2) → (x1^2 + x2^2 + 8) / 10 :
  g2 := (x1, x2) → (x1*x2^2 + x1 + 8) / 10 :
  # Set tolerance and maximum iterations
  tol := 1e-5 :
  max_iter := 100 :
  # Initial guess
  x0 := [0.5, 0.5] :
  x_old := x0 :
  for k from 1 to max_iter do
    x_new := [ g1(x_old[1], x_old[2]), g2(x_old[1], x_old[2]) ] :
    if abs(x_new[1] − x_old[1]) < tol and abs(x_new[2] − x_old[2]) < tol then
      printf("Fixed-point iteration converged in %d steps\n", k) :
      break:
    end if;
    x_old := x_new :
  end do:
  x_fixed := x_new :
  printf("Solution (Fixed-point) = [%f, %f]\n", x_fixed[1], x_fixed[2]);
  # (c) Gauss-Seidel iteration
  x_old := x0 :
  for k from 1 to max_iter do
    x1_new := g1(x_old[1], x_old[2]) :
    x2_new := g2(x1_new, x_old[2]) : # use updated x1 for faster convergence
    if abs(x1_new − x_old[1]) < tol and abs(x2_new − x_old[2]) < tol then
      printf("Gauss-Seidel iteration converged in %d steps\n", k) :
      break:
    end if;
    x_old := [x1_new, x2_new] :
  end do:

  x_gs := [x1_new, x2_new] :
  printf("Solution (Gauss-Seidel) = [%f, %f]\n", x_gs[1], x_gs[2]);

Fixed-point iteration converged in 12 steps
Solution (Fixed-point) = [0.999995, 0.999995]
Gauss-Seidel iteration converged in 10 steps
Solution (Gauss-Seidel) = [0.999996, 0.999998]

=
```